Caminhos mínimos 6

Além da orientação nos arcos, os grafos desta seção têm peso nas arcos, ou seja, esses grafos são definidos por uma tripla (V, E, ρ) com $\rho : E \to \mathbb{R}_+$ (escrevemos $\rho(u,t)$ em vez de $\rho\big((u,t)\big)$). O ***comprimento*** de um caminho orientado $P = x_1, x_2, \dots, x_k$ é definido, naturalmente, como a soma dos pesos (comprimentos) das arcos nesse caminho

$$c(P) = \sum_{i=1}^{k-1} \rho(x_i, x_{i+1}),$$

e a ***distância*** entre dois vértices é o comprimento do menor caminho orientado que os liga,

$$\operatorname{dist}_G(u,v) = \min\big\{c(P) : P \text{ é um } (u \to v)\text{-caminho}\big\}$$

quando existe algum caminho, caso contrário convencionamos que $\operatorname{dist}_G(u,v) = \infty$ com as convenções adotadas na seção 1.3. Um $(u \to v)$-caminho de comprimento $\operatorname{dist}_G(u,v)$ é chamado de ***caminho mínimo***.

O seguinte diagrama representa um grafo dirigido com pesos nas arcos

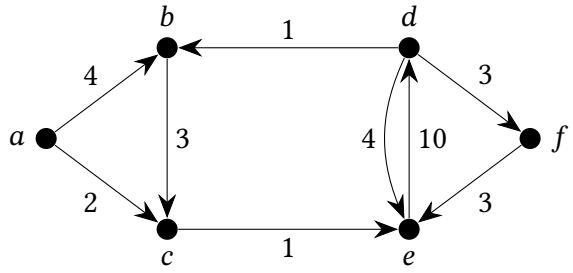


Figura 6.1: Diagrama de um grafo dirigido com peso nas arcos. Note que $\operatorname{dist}(c,e) \neq \operatorname{dist}(e,c)$, $\operatorname{dist}(a,f) = 10$.

Em um grafo dirigido G podemos ter

$$\operatorname{dist}_G(u,v) \neq \operatorname{dist}_G(v,u).$$

Para um grafo dirigido G qualquer, com pesos nos arcos, e s um vértice de G as seguintes propriedades são satisfeitas:

***desigualdade triangular*:** vale a seguinte variante da desigualdade triangular (1.6): para todo arco $(v,u) \in E(G)$

$$\operatorname{dist}_G(s,u) \leq \operatorname{dist}_G(s,v) + \rho(v,u). \tag{6.1}$$

***subestrutura ótima*:** Se $P = v_1, v_2, \dots, v_k$ é um caminho mínimo num grafo G então $P_{i,j} = v_i, \dots, v_j$ é um caminho mínimo com extremos v_i e v_j em G para quaisquer i, j com $1 \leq i < j \leq k$.

6.1 Algoritmo de Dijkstra

O ***algoritmo de Dijkstra*** foi concebido por Edsger W. Dijkstra em 1956 e publicado em 1959 na nota *A note on two problems in connexion with graphs*, na *Numerische Mathematik*. Segundo o próprio Dijkstra, ele o inventou em cerca de vinte minutos, sem papel e lápis, pensando num exemplo para demonstrar o computador ARMAC[1] a um público sem formação técnica: o menor caminho entre duas cidades de um mapa rodoviário. A mesma nota trata traz o algoritmo de Prim que estudamos na Seção 9.

Há registros de que outros autores chegaram a ideias próximas na mesma época.

1: *Automatic Relay and Magnetic-drum Automatic Calculator*, foi um computador Holandês construído em 1956 no Centro Matemático de Amsterdã, atual *Centrum Wiskunde & Informatica* (CWI), prestigioso centro de pesquisa na área de matemática e ciência da computação teórica.

São dados um grafo dirigido $G = (V, E)$, pesos $\rho : E \to \mathbb{R}_+$ e um vértice $s \in V$. Queremos determinar $\text{dist}_G(s, v)$ para todo $v \in V$. Para isso, usaremos um vetor d[ ] indexado pelos vértices, um vetor p[ ] de predecessores e um subconjunto $S \subseteq V$ de modo que, em qualquer instante da execução do algoritmo,

S: o conjunto S contém somente os vértices $v \in V$ que já têm a distância $\text{dist}_G(s, v)$ computada corretamente;

d[ ]: para todo $v \in V$,

- se $v \in S$, então $\text{d}[v] = \text{dist}_G(s, v)$;
- se $v \in \overline{S} = V \setminus S$ e $\text{d}[v] < \infty$, então $\text{d}[v] = c(P)$ para algum $(s \to v)$-caminho orientado P;
- se $v \in \overline{S}$ e v ainda não foi alcançado, então $\text{d}[v] = \infty$.

O algoritmo que descrevemos a seguir recebe G e s como acima e funciona da seguinte forma.

1. De início $\text{d}[s] = 0$, pois s está a distância 0 dele mesmo, $S = \varnothing$ e $\text{d}[v] = \infty$ para todo $v \neq s$. Assim, na primeira iteração o vértice escolhido é s, o único com d finito.
2. Enquanto $\overline{S} \neq \varnothing$, retiramos de $\overline{S}$ o vértice u de menor valor de d[ ] e o colocamos em S. Provaremos a seguir que u é o vértice de $\overline{S}$ mais próximo de s e que, portanto, $\text{d}[u] = \text{dist}_G(s, u)$.
3. Em seguida, visitamos cada sucessor $t \in N_G^+(u)$. Como $\text{d}[u] = \text{dist}_G(s, u)$, a desigualdade triangular diz que $\text{dist}_G(s, t) \leq \text{d}[u] + \rho(u, t)$; testamos então se um caminho de custo menor até t, passando por u, foi encontrado e, em caso afirmativo, atualizamos $\text{d}[t]$ e $\text{p}[t]$. Essa etapa é chamada de ***relaxação*** do arco (u, t):

Algoritmo 31: Relaxação(u, t).

Dado : um arco $(u, t) \in E(G)$.
Devolve : atualiza $\text{d}[t]$ e $\text{p}[t]$ se um caminho de menor custo até t foi encontrado passando por u.

```
1 se d[t] > d[u] + ρ(u,t) então
2     d[t] ← d[u] + ρ(u,t);
3     p[t] ← u;
```

Notemos que a condição na Relaxação(u,t) é sempre falsa para $t \in S$: se u acabou de entrar em S, então $\mathrm{d}[t] = \mathrm{dist}_G(s,t) \leq \mathrm{dist}_G(s,u) + \rho(u,t) = \mathrm{d}[u] + \rho(u,t)$. O algoritmo prossegue até que $S = V$.

O seguinte algoritmo é conhecido como ***algoritmo de Dijkstra***.

Algoritmo 32: Dijkstra(G,s).

Dado : um grafo dirigido G, pesos $\rho(e) \geq 0$ nos arcos e um vértice $s \in V(G)$.

Devolve : $\mathrm{d}[v] = \mathrm{dist}_G(s,v)$ para todo $v \in V(G)$ e um vetor p de predecessores.

```
para cada v ∈ V(G) faça
    d[v] ← ∞;
    p[v] ← nil;
d[s] ← 0;
S ← ∅;
enquanto S̄ ≠ ∅ faça
    seja u tal que d[u] = min{d[v] : v ∈ S̄};
    insira u em S;
    para cada t ∈ N⁺_G(u) faça
        Relaxação(u, t);
retorne d, p
```

Uma demonstração formal de que o algoritmo funciona, isto é, no final $\mathrm{d}[v] = \mathrm{dist}_G(s,v)$ para todo $v \in V$, e a complexidade do algoritmo são determinadas a seguir.

Análise e correção do algoritmo de Dijkstra

Vamos mostrar que o algoritmo funciona corretamente.

Proposição 6.1

Em qualquer momento da execução de Dijkstra(G,s) valem os seguintes fatos.

(a) $\mathrm{d}[v] \geq \mathrm{dist}_G(s,v)$ para todo $v \in V$ e, se $\mathrm{d}[v] = \mathrm{dist}_G(s,v)$, o valor de $\mathrm{d}[v]$ não muda.
(b) Se $\mathrm{dist}_G(s,v) = \infty$ então $\mathrm{d}[v] = \infty$.

Demonstração. Para provar (a) vamos supor o contrário e assumir que $w \in V$ é o primeiro vértice, em ordem temporal, para o qual ocorreu

$$\mathrm{d}[w] < \mathrm{dist}_G(s,w) \tag{6.2}$$

durante uma execução, e seja $(u,w) \in E$ o arco cuja relaxação causou a mudança no valor de $\mathrm{d}[w]$. Assim a relaxação faz

$$\mathrm{d}[w] = \mathrm{d}[u] + \rho(u,w) \tag{6.3}$$

e, como w foi o primeiro caso de (6.2), nesse momento $\mathrm{d}[u] \geq \mathrm{dist}_G(s,u)$; logo

$$\mathrm{d}[u] + \rho(u,w) \geq \mathrm{dist}_G(s,u) + \rho(u,w). \tag{6.4}$$

Das três relações (6.2), (6.3) e (6.4) deduzimos

$$\begin{aligned} \mathrm{dist}_G(s,w) &> \mathrm{d}[u] + \rho(u,w) \\ &\geq \mathrm{dist}_G(s,u) + \rho(u,w), \end{aligned}$$

contradizendo a desigualdade triangular. Ademais, uma relaxação nunca aumenta o valor $\mathrm{d}[v]$; logo, uma vez que $\mathrm{d}[v] = \mathrm{dist}_G(s,v)$, esse valor não muda, pois não pode diminuir.

Para provar (b) notemos que, por (a), $\mathrm{d}[v] \geq \mathrm{dist}_G(s,v) = \infty$; portanto nunca será atribuído um valor real a $\mathrm{d}[v]$. □

Proposição 6.2

Seja $P = x_0, x_1, \ldots, x_k, v$ um caminho mínimo de $x_0 = s$ a v em G, com $k \geq 0$. Se $\mathrm{d}[x_k] = \mathrm{dist}_G(s,x_k)$ então após Relaxação(x_k, v) teremos $\mathrm{d}[v] = \mathrm{dist}_G(s,v)$.

Demonstração. Após Relaxação(x_k, v) temos

$$\mathrm{d}[v] \leq \mathrm{d}[x_k] + \rho(x_k, v)$$

e por hipótese

$$\mathrm{d}[x_k] + \rho(x_k, v) = \mathrm{dist}_G(s,x_k) + \rho(x_k, v),$$

logo

$$\mathrm{d}[v] \leq \mathrm{dist}_G(s,x_k) + \rho(x_k, v).$$

Pela subestrutura ótima, $x_0, \ldots, x_k$ é um caminho mínimo até x_k e

$$\mathrm{dist}_G(s,x_k) + \rho(x_k, v) = \mathrm{dist}_G(s,v),$$

portanto $\mathrm{d}[v] \leq \mathrm{dist}_G(s,v)$. Pela Proposição 6.1(a), $\mathrm{d}[v] \geq \mathrm{dist}_G(s,v)$, portanto $\mathrm{d}[v] = \mathrm{dist}_G(s,v)$. □

A correção do algoritmo segue do próximo resultado. Provaremos um *invariante* do laço enquanto. Esse invariante é uma propriedade que permanece válida a cada iteração do laço, independentemente do número de vezes que é executado.

Teorema 6.1

Antes de cada teste da condição do laço **enquanto** na linha 6 (inclusive o último, quando o laço termina) vale que

$$\mathrm{d}[v] = \mathrm{dist}_G(s,v) \text{ para todo } v \in S.$$

Demonstração. Suponha que o invariante falha em algum momento e seja r o menor número de iterações completas após as quais existe $v \in S$ com $\mathrm{d}[v] \neq \mathrm{dist}_G(s, v)$. Para $r = 0$ temos $S = \emptyset$ e após a primeira iteração temos $S = \{s\}$ e $\mathrm{d}[s] = 0 = \mathrm{dist}_G(s, s)$; logo $r \geq 2$.

Seja u o vértice inserido em S na r-ésima iteração. Pela minimalidade de r, os vértices de S antes dessa iteração têm d correto e, pela Proposição 6.1(a), esse valor não muda; a relaxação de arcos que saem de u só altera d de sucessores de u, e u não tem laços. Logo $\mathrm{d}[u] \neq \mathrm{dist}_G(s, u)$ no instante em que u é escolhido na linha 7, e nesse instante, como $r \geq 2$, temos $s \in S$ (o primeiro vértice inserido é s, o único com d finito) e $u \in \overline{S}$. Por 6.1(a),

$$\mathrm{d}[u] > \mathrm{dist}_G(s, u). \tag{6.5}$$

Por 6.1(b), $\mathrm{dist}_G(s, u) < \infty$, e existe um caminho mínimo, digamos que $P = x_0, x_1, \ldots, x_m, x_{m+1}$ em G com $x_0 = s$ e $x_{m+1} = u$.

Como $x_0 \in S$ e $x_{m+1} \notin S$, seja i o menor índice, $0 \leq i \leq m$, tal que $x_{i+1} \notin S$; então $x_0, \ldots, x_i \in S$ e (x_i, x_{i+1}) é o primeiro arco de P que sai de S. Notemos que é possível termos $x_i = s$ e $x_{i+1} = u$.

Quando x_i entrou em S, em uma iteração anterior à r-ésima, tínhamos $\mathrm{d}[x_i] = \mathrm{dist}_G(s, x_i)$ e, logo em seguida, ocorreu Relaxação(x_i, x_{i+1}). Como $x_0, \ldots, x_i, x_{i+1}$ é um caminho mínimo, a Proposição 6.2 dá $\mathrm{d}[x_{i+1}] = \mathrm{dist}_G(s, x_{i+1})$ após essa relaxação, e por 6.1(a) esse valor não muda. Além disso, $\mathrm{dist}_G(s, x_{i+1}) \leq \mathrm{dist}_G(s, u)$, pois os pesos são não-negativos. Usando (6.5),

$$\mathrm{d}[x_{i+1}] = \mathrm{dist}_G(s, x_{i+1}) \leq \mathrm{dist}_G(s, u) < \mathrm{d}[u].$$

Mas x_{i+1} e u pertencem a $\overline{S}$ e u foi escolhido com $\mathrm{d}[u]$ mínimo em $\overline{S}$, de modo que $\mathrm{d}[u] \leq \mathrm{d}[x_{i+1}]$, uma contradição. □

A correção do algoritmo de Dijkstra segue facilmente do teorema acima: cada iteração remove um vértice de $\overline{S}$ e, portanto, após $|V|$ iterações $\overline{S} = \emptyset$, ou seja $S = V(G)$, e o teorema, aplicado ao teste final do laço, garante que $\mathrm{d}[v] = \mathrm{dist}_G(s, v)$ para todo $v \in V(G)$. Os vértices não alcançáveis a partir de s terminam com $\mathrm{d}[v] = \infty$ pela Proposição 6.1(b).

Agora, vamos analisar a complexidade do algoritmo de Dijkstra. As linhas de inicialização tomam tempo $O(|V|)$ e cada Relaxação(u, t) toma tempo constante. A complexidade de tempo do laço depende de como é determinado o mínimo na linha 7. O laço da linha 6 é executado uma vez para cada $u \in V(G)$; a sua contribuição para a complexidade do algoritmo é

$$\sum_{u \in V} \Big(T_7(u) + T_8(u) + T_{9,10}(u) \Big)$$

onde $T_i(u)$ é o tempo gasto na linha i na iteração em que u é escolhido

na linha 7. Se o mínimo é determinado fazendo comparações, então em cada rodada o custo é $|\overline{S}|-1$ e $|\overline{S}|$ assume os valores $|V|, |V|-1, \ldots, 1$; portanto,

$$\sum_{u\in V} T_7(u) = \sum_{i=1}^{|V|}(i-1) = \frac{|V|(|V|-1)}{2} = O(|V|^2).$$

Ademais $\sum_{u\in V} T_8(u) = O(|V|)$ e, como cada arco (u,t) é relaxado uma única vez, na iteração em que u é escolhido,

$$\sum_{u\in V} T_{9,10}(u) = \sum_{u\in V}\sum_{t\in N_G^+(u)} O(1) = O(|E|).$$

Logo o tempo total, incluindo a inicialização, é

$$O(|V|^2 + |E|),$$

para grafos dirigidos simples.

Outra possibilidade é usar uma fila de prioridades para determinar o mínimo

Mantendo os vértices de $\overline{S}$ em uma heap binária com chaves d[ ] e o vetor auxiliar pos[ ], como no algoritmo de Dijkstra–Jarník–Prim com heap (Seção 9), cada escolha do mínimo na linha 7 é uma operação `extrair-min` de custo $O(\log|V|)$ e cada relaxação que melhora d[t] é uma operação `diminuir-chave` de custo $O(\log|V|)$. Como há $|V|$ extrações, no máximo $|E|$ relaxações bem-sucedidas e a construção inicial da heap custa $O(|V|)$, o algoritmo de Dijkstra tem complexidade de tempo $O\big((|V|+|E|)\log|V|\big)$.

6.2 Algoritmo de Bellman–Ford

O ***algoritmo de Bellman–Ford***, ou ***algoritmo de Bellman–Ford–Moore*** leva o nome dos matemáticos Richard Bellman, Lester R. Ford Jr. e Edward F. Moore, que o publicaram em 1958, 1956 e 1959, respectivamente, embora já tenha sido proposto em 1955 por Alfonso Shimbel.

É um algoritmo anterior e mai lentos que o algoritmo de Dijkstra, porém permite pesos negativos nos arcos. Como cada vértice só precisa dos valores de seus vizinhos, o algoritmo tem uma versão distribuída, que está na base dos protocolos de roteamento por vetor de distâncias, como o RIP (*Routing Information Protocol*), usados em redes de computadores.

Seja $G = (V, E, \rho)$ um grafo dirigido com pesos nos arcos, que podem ser negativos. Um ***circuito negativo*** em G é um circuito orientado em que a soma dos pesos dos arcos é negativa. Num circuito negativo podemos ficar dando voltas e a cada volta o comprimento diminui; assim, uma tentativa ingênua de computar distâncias em G

Bellman foi um dos criadores do conceito de *programação dinâmica*, um paradigma para algoritmos de otimização e também cunhou o termo "maldição da dimensionalidade", muito em voga atualmente com os avanços do aprendizado de máquina. Ford, especialista em fluxo em redes, escreveu junto com D.R. Fulkerson uma solução sobre o problema do fluxo máximo e o algoritmo de Ford-Fulkerson, publicado como um relatório técnico em 1954 e em um periódico em 1956, estabeleceu o teorema do fluxo máximo e do corte mínimo; ambos, trabalhavam na *RAND Corporation*. Moore foi inventor da máquina de estados finitos de Moore e um dos pioneiros da vida artificial.

poderia facilmente entrar em um laço infinito.

O algoritmo de Bellman–Ford recebe G e um vértice s e devolve *falso* se G tem um circuito negativo alcançável a partir de s, e *verdadeiro* caso contrário. Neste último caso, os valores $\mathrm{d}[v]$ calculados são as distâncias $\mathrm{dist}_G(s, v)$, com ∞ para os vértices não alcançáveis a partir de s.

A ideia do algoritmo é a seguinte. Suponha que não há circuito negativo alcançável a partir de s e seja v um vértice alcançável. Então existe um caminho mínimo

$$P = x_0, x_1, \ldots, x_k, \qquad x_0 = s,\ x_k = v,$$

e, como um caminho não repete vértices, $k \leq |V| - 1$. Repetimos $|V| - 1$ vezes a Relaxação (Algoritmo 31) de todos os arcos de G, em qualquer ordem. Na primeira rodada o arco (x_0, x_1) é relaxado e, depois dela, $\mathrm{d}[x_1] = \mathrm{dist}_G(s, x_1)$; na segunda rodada o arco (x_1, x_2) é relaxado e, depois dela, $\mathrm{d}[x_2] = \mathrm{dist}_G(s, x_2)$; e assim por diante: depois da rodada i temos $\mathrm{d}[x_i] = \mathrm{dist}_G(s, x_i)$. Como todo caminho tem no máximo $|V| - 1$ arcos, ao final das repetições as distâncias estão determinadas.

Para detectar um circuito negativo alcançável basta testar se há algum arco $(u, w) \in E(G)$ com $\mathrm{d}[w] > \mathrm{d}[u] + \rho(u, w)$ depois das $|V| - 1$ rodadas.

Algoritmo 33: Bellman–Ford(G, s).

Dado : $G = (V, E, \rho)$ dirigido com pesos ρ e $s \in V(G)$.

Devolve : *falso* se G tem um circuito negativo alcançável a partir de s; caso contrário, *verdadeiro*, e então $\mathrm{d}[v] = \mathrm{dist}_G(s, v)$ para todo $v \in V(G)$ e p é um vetor de predecessores.

```
1  para cada v ∈ V(G) faça
2      d[v] ← ∞;
3      p[v] ← nil;
4  d[s] ← 0;
5  cont ← 1;
6  enquanto cont ≤ |V(G)| − 1 faça
7      para cada (u, w) ∈ E(G) faça
8          Relaxação(u, w);
9      cont ← cont + 1;
10 para cada (u, w) ∈ E(G) faça
11     se d[w] > d[u] + ρ(u, w) então
12         retorne falso;
13 retorne verdadeiro, d, p
```

Análise e correção do algoritmo de Bellman–Ford

O resultado a seguir prova que o algoritmo devolve corretamente o valor booleano prometido e que, no caso *verdadeiro*, os valores $\mathrm{d}[v]$ são as distâncias.

Lema 6.1

Seja $G = (V, E, \rho)$ um grafo dirigido com pesos e $s \in V$. *Bellman–Ford*(G, s) devolve *falso* se, e somente se, G tem um circuito negativo alcançável a partir de s. Além disso, se devolve *verdadeiro*, então $\mathrm{d}[v] = \mathrm{dist}_G(s, v)$ para todo $v \in V$.

Demonstração. Valem duas afirmações, quaisquer que sejam os pesos.

(a) Em todo momento, $\mathrm{d}[v] = \infty$ ou $\mathrm{d}[v]$ é o peso de algum passeio de s a v. De fato, isso vale no início e a Relaxação(u, w) só atribui $\mathrm{d}[w] \leftarrow \mathrm{d}[u] + \rho(u, w)$, que é o peso do passeio de s a u seguido do arco (u, w).

(b) Seja $P = x_0, x_1, \ldots, x_k$ um caminho com $x_0 = s$ e $k \leq |V| - 1$. Ao final da rodada i (a i-ésima execução do laço da linha 6), $\mathrm{d}[x_i] \leq \rho(x_0, \ldots, x_i)$, o peso do trecho de P de x_0 até x_i. Provamos por indução em i. Para $i = 0$, $\mathrm{d}[s] \leq 0$ pois $\mathrm{d}[s]$ é inicializado com 0. Os valores d nunca aumentam. Assim, quando (x_{i-1}, x_i) é relaxado na rodada i, temos $\mathrm{d}[x_{i-1}] \leq \rho(x_0, \ldots, x_{i-1})$ (hipótese de indução) e, depois da relaxação, $\mathrm{d}[x_i] \leq \mathrm{d}[x_{i-1}] + \rho(x_{i-1}, x_i) \leq \rho(x_0, \ldots, x_i)$. Em particular, $\mathrm{d}[v] < \infty$ para todo v alcançável a partir de s, pois existe um caminho de s a v e ele tem no máximo $|V| - 1$ arcos.

Se há um circuito negativo alcançável, o algoritmo devolve falso. Seja $C = v_0, v_1, \ldots, v_{k-1}, v_0$ um circuito negativo alcançável a partir de s e $v_k := v_0$. Todos os v_i são alcançáveis, logo $\mathrm{d}[v_i] < \infty$ por (b). Suponha que o algoritmo devolve *verdadeiro*. Então $\mathrm{d}[v_i] \leq \mathrm{d}[v_{i-1}] + \rho(v_{i-1}, v_i)$ para $i = 1, \ldots, k$ e, somando sobre os k arcos de C,

$$\sum_{i=1}^{k} \mathrm{d}[v_i] \leq \sum_{i=1}^{k} \mathrm{d}[v_{i-1}] + \sum_{i=1}^{k} \rho(v_{i-1}, v_i).$$

Como $v_k = v_0$, as duas somas de d são iguais e finitas; logo $0 \leq \sum_{i=1}^{k} \rho(v_{i-1}, v_i)$, contradizendo o fato de C ser negativo.

Se não há circuito negativo alcançável, o algoritmo devolve verdadeiro e $\mathrm{d}[v] = \mathrm{dist}_G(s, v)$. Todo passeio de s a v é a união de um caminho de s a v com circuitos alcançáveis a partir de s, todos de peso ≥ 0; assim, seu peso é $\geq \mathrm{dist}_G(s, v)$. Por (a), $\mathrm{d}[v] \geq \mathrm{dist}_G(s, v)$ sempre. Por outro lado, se v é alcançável, existe um caminho mínimo de s a v, com $k \leq |V| - 1$ arcos, e por (b) com $i = k$ temos $\mathrm{d}[v] \leq \mathrm{dist}_G(s, v)$ ao final das $|V| - 1$ rodadas. Se v não é alcançável, $\mathrm{d}[v] = \infty = \mathrm{dist}_G(s, v)$ por (a). Portanto $\mathrm{d}[v] = \mathrm{dist}_G(s, v)$ para todo $v \in V$. Finalmente, para cada arco (u, w) com u alcançável, $\mathrm{d}[w] = \mathrm{dist}_G(s, w) \leq \mathrm{dist}_G(s, u) + \rho(u, w) = \mathrm{d}[u] + \rho(u, w)$, pois

um caminho mínimo até u seguido de (u, w) é um passeio, de peso $\geq \text{dist}_G(s, w)$; e se u não é alcançável, $\text{d}[u] = \infty$. Em ambos os casos o teste do último laço falha e o algoritmo devolve *verdadeiro*. □

Para a complexidade, a inicialização custa $\Theta(|V|)$, as $|V|-1$ rodadas das linhas 6–7 fazem $|E|$ relaxações de tempo $O(1)$ cada, e a verificação final custa $O(|E|)$. O tempo total é $O(|V| + |V||E|)$, que é $O(|V||E|)$ se $|E| \geq 1$.

6.3 Algoritmo de Floyd–Warshall

Os algoritmos de Dijkstra e de Bellman–Ford calculam as distâncias a partir de *um* vértice s. Para obter as distâncias entre *todos os pares* de vértices poderíamos executar um deles $|V|$ vezes, uma para cada origem. O ***algoritmo de Floyd–Warshall*** faz isso de forma direta, com três laços aninhados e uma matriz.

O algoritmo foi publicado por Robert W. Floyd em 1962, na seção de algoritmos da *Communications of the ACM* (***Algorithm 97: Shortest path***), a partir de um teorema de Stephen Warshall, do mesmo ano, sobre o fecho transitivo de matrizes booleanas, isto é, sobre a alcançabilidade em grafos dirigidos (Exercício 113). A mesma estrutura de três laços já aparecia em trabalhos anteriores, como os de Kleene (1956), sobre expressões regulares, e de Bernard Roy (1959), sobre fecho transitivo, e por isso o método também é chamado de algoritmo de Roy–Floyd–Warshall ou, simplesmente, algoritmo de Floyd. Usa o paradigma de programação dinâmica.

Seja $G = (V, E, \rho)$ um grafo dirigido simples, com $V = \{1, 2, \ldots, n\}$ e pesos ρ nos arcos, que podem ser negativos. Como antes, $\text{dist}_G(i, j)$ é o menor peso de um $(i \to j)$-caminho orientado, com $\text{dist}_G(i, i) = 0$ e $\text{dist}_G(i, j) = \infty$ se j não é alcançável a partir de i. Queremos determinar $\text{dist}_G(i, j)$ para todo par $(i, j) \in V \times V$ ou descobrir que G tem um circuito negativo.

Os ***vértices internos*** de um caminho $x_0, \ldots, x_m$ são $x_1, \ldots, x_{m-1}$ (um arco não tem vértices internos). A ideia do algoritmo é considerar, para $k = 0, 1, \ldots, n$, só os caminhos cujos vértices internos pertencem a $\{1, \ldots, k\}$. Seja P um $(i \to j)$-caminho mínimo entre esses. Há duas possibilidades:

- k não é vértice interno de P; então os vértices internos de P estão em $\{1, \ldots, k-1\}$;
- k é vértice interno de P; como P é um caminho, passa por k uma única vez, e P é a união de um $(i \to k)$-caminho e de um $(k \to j)$-caminho, ambos com vértices internos em $\{1, \ldots, k-1\}$, e cada um deles é mínimo com essa propriedade (subestrutura ótima, como em (F2) na página 86).

Assim, se $\text{d}_k[i, j]$ denota o peso de um tal caminho mínimo (entre os que têm vértices internos em $\{1, \ldots, k\}$), então, na ausência de circuitos negativos,

$$\text{d}_k[i, j] = \min\{\text{d}_{k-1}[i, j],\ \text{d}_{k-1}[i, k] + \text{d}_{k-1}[k, j]\},$$

com $\text{d}_0[i, j] = \rho(i, j)$ se $(i, j) \in E$, $\text{d}_0[i, i] = 0$ e $\text{d}_0[i, j] = \infty$ nos demais casos; e $\text{dist}_G(i, j) = \text{d}_n[i, j]$. O algoritmo a seguir mantém uma única matriz d[,], que ao final da iteração k contém d_k, e uma matriz p[,] de predecessores: p$[i, j]$ é o penúltimo vértice do caminho encontrado de i a j.

Algoritmo 34: Floyd–Warshall(G).

Dado : $G = (V, E, \rho)$ um grafo dirigido simples e pesos ρ pesos nos arcos.

Devolve : *falso* se G tem um circuito negativo; caso contrário, *verdadeiro*, e então $\mathrm{d}[i, j] = \mathrm{dist}_G(i, j)$ para todo $i, j \in V$ e p é a matriz de predecessores.

```
1  para cada i ∈ V(G) faça
2      para cada j ∈ V(G) faça
3          d[i, j] ← ∞;
4          p[i, j] ← nil;
5      d[i, i] ← 0;
6  para cada (i, j) ∈ E(G) faça
7      d[i, j] ← ρ(i, j);
8      p[i, j] ← i;
9  para cada k = 1, 2, ... , n, nessa ordem faça
10     para cada i ∈ V(G) faça
11         para cada j ∈ V(G) faça
12             se d[i, k] + d[k, j] < d[i, j] então
13                 d[i, j] ← d[i, k] + d[k, j];
14                 p[i, j] ← p[k, j];
15 para cada i ∈ V(G) faça
16     se d[i, i] < 0 então
17         retorne falso;
18 retorne verdadeiro, d, p
```

Notemos que o teste da linha 12 é falso se $\mathrm{d}[i, k] = \infty$ ou $\mathrm{d}[k, j] = \infty$, pela convenção acima; e que, na iteração k, a atualização de $\mathrm{d}[i, j]$ usa valores de $\mathrm{d}[i, k]$ e $\mathrm{d}[k, j]$ que podem ter sido atualizados na própria iteração k. A prova a seguir mostra que isso não prejudica a correção.

Análise e correção do algoritmo de Floyd–Warshall

Lema 6.2

Floyd–Warshall(G) devolve *falso* se, e somente se, G tem um circuito negativo. Além disso, se devolve *verdadeiro*, então $\mathrm{d}[i, j] = \mathrm{dist}_G(i, j)$ para todo $i, j \in V$.

Demonstração. Valem duas afirmações, quaisquer que sejam os pesos.

(a) Em todo momento durante e depois da iteração k, $\mathrm{d}[i, j] = \infty$ ou $\mathrm{d}[i, j]$ é o peso de algum $(i \to j)$-passeio com vértices internos em $\{1, \dots, k\}$. De fato, isso vale antes da primeira iteração ($k = 0$),

pois os valores iniciais são pesos de passeios triviais ou de arcos. E a atribuição $\mathrm{d}[i,j] \leftarrow \mathrm{d}[i,k] + \mathrm{d}[k,j]$ é o peso de um $(i \to k)$-passeio seguido de um $(k \to j)$-passeio, que é um $(i \to j)$-passeio cujos vértices internos estão em $\{1, \ldots, k\}$.

(b) Ao final da iteração k, $\mathrm{d}[i,j] \leq c(P)$ para todo par (i,j) e todo $(i \to j)$-caminho P cujos vértices internos estão em $\{1, \ldots, k\}$. Provamos por indução em k. Para $k = 0$, P é um arco, ou o caminho trivial, e o valor inicial de $\mathrm{d}[i,j]$ é $\rho(i,j)$ ou 0. Para $k \geq 1$, lembre que os valores de d nunca aumentam. Se k não é vértice interno de P, então vale a hipótese de indução. Senão, P passa por k uma única vez e é a união de um $(i \to k)$-caminho P_1 e de um $(k \to j)$--caminho P_2 com vértices internos em $\{1, \ldots, k-1\}$. Pela hipótese de indução, no início da iteração k temos $\mathrm{d}[i,k] \leq c(P_1)$ e $\mathrm{d}[k,j] \leq c(P_2)$, e esses valores não aumentam; logo, depois do teste do par (i,j), $\mathrm{d}[i,j] \leq \mathrm{d}[i,k] + \mathrm{d}[k,j] \leq c(P)$.

Se G não tem circuito negativo, todo passeio fechado tem peso ≥ 0 e todo $(i \to j)$-passeio é a união de um $(i \to j)$-caminho com circuitos, portanto tem peso $\geq \mathrm{dist}_G(i,j)$. Por (a), $\mathrm{d}[i,j] \geq \mathrm{dist}_G(i,j)$ ao final; em particular $\mathrm{d}[i,i] \geq 0$, e como $\mathrm{d}[i,i]$ começa em 0 e não aumenta, vale $\mathrm{d}[i,i] = 0$ e o algoritmo devolve *verdadeiro*. Por (b) com $k = n$, todo caminho de i a j tem peso $\geq \mathrm{d}[i,j]$, logo $\mathrm{d}[i,j] \leq \mathrm{dist}_G(i,j)$ (trivial se $\mathrm{dist}_G(i,j) = \infty$). Portanto $\mathrm{d}[i,j] = \mathrm{dist}_G(i,j)$.

Se G tem um circuito negativo, seja C um circuito negativo. Como G não tem laços, C tem pelo menos dois vértices. Seja k o maior vértice de C e $j \neq k$ outro vértice de C. Então C é a união de um $(j \to k)$-caminho P_1 e de um $(k \to j)$-caminho P_2, ambos com vértices internos em $\{1, \ldots, k-1\}$. Por (b) (iteração $k-1$), no início da iteração k temos $\mathrm{d}[j,k] \leq c(P_1)$ e $\mathrm{d}[k,j] \leq c(P_2)$, logo, após o teste do par (j,j) na iteração k, $\mathrm{d}[j,j] \leq c(P_1) + c(P_2) = c(C) < 0$. Como $\mathrm{d}[j,j]$ não aumenta, o último laço encontra $\mathrm{d}[j,j] < 0$ e o algoritmo devolve *falso*. Isso prova as duas implicações, pois se G não tem circuito negativo o algoritmo devolve *verdadeiro*, como visto acima. □

Para a complexidade, os três laços aninhados das linhas 9–11 executam n^3 vezes um teste e, possivelmente, duas atribuições de tempo $O(1)$; a inicialização e o último laço custam $O(n^2 + |E|) = O(n^2)$. O tempo total é $\Theta(n^3)$ e o espaço usado pelas matrizes é $\Theta(n^2)$. Em comparação, executar o algoritmo de Dijkstra $|V|$ vezes, quando $\rho \geq 0$, custa $O(|V|^3)$ com a versão simples e $O(|V||E| \log |V|)$ com *heap* (que é melhor para grafos esparsos), e executar Bellman–Ford $|V|$ vezes custa $O(|V|^2|E|)$, que é $O(|V|^4)$ para grafos densos. O algoritmo de Floyd–Warshall aceita pesos negativos, detecta circuitos negativos e é muito simples de implementar.

Exercícios

Exercício 98. Mostre que o vetor p[ ] devolvido por Dijkstra(G, s) permite reconstruir caminhos mínimos: para todo v alcançável a partir de s, a sequência v, p[v], p[p[v]], ... termina em s e, lida de trás para frente, é um $(s \to v)$-caminho orientado de custo $\mathrm{dist}_G(s, v)$.

Exercício 99. Mostre, com um exemplo, que o algoritmo de Dijkstra pode devolver valores incorretos se G tem arcos de peso negativo. (Dica: G com arcos (s, a), (s, b), (b, a), (a, c) de pesos $2, 3, -2, 1$.) Em que ponto exato da demonstração do Teorema 6.1 a hipótese $\rho \geq 0$ é usada?

Exercício 100. Seja G um grafo dirigido com pesos $\rho(e) \geq 0$ nos arcos. Suponha que $v_1, v_2, \ldots, v_k$ é um caminho orientado de custo mínimo de v_1 a v_k em G. Prove que $v_i, \ldots, v_j$ é um caminho orientado de custo mínimo de v_i a v_j em G para quaisquer i, j com $1 \leq i < j \leq k$. (Dica: se existisse um caminho mais barato Q de v_i a v_j, trocar o trecho por Q daria um passeio de v_1 a v_k de custo menor; retire os circuitos.) Mostre que o resultado vale também se G tem arcos de peso negativo, mas nenhum circuito orientado de custo negativo.

Exercício 101. Seja G um grafo dirigido com pesos $\rho(e) \geq 0$ nos arcos e $s \in V(G)$. Prove que, para todo arco $(u, w) \in E(G)$,

$$\mathrm{dist}_G(s, w) \leq \mathrm{dist}_G(s, u) + \rho(u, w)$$

(equação (6.1)). Onde a hipótese $\rho \geq 0$ é usada? Mostre que a desigualdade pode falhar se G tem um circuito orientado de custo negativo. (Dica: arcos (s, w), (w, u), (u, w) de pesos $1, -3, 1$.)

Exercício 102. Construa um grafo dirigido, com pesos ρ nos arcos, alguns negativos mas sem circuito orientado de custo negativo, no qual o algoritmo Dijkstra(G, s) devolve $\mathrm{d}[v] \neq \mathrm{dist}_G(s, v)$ para algum v. Em que ponto exato da demonstração do Teorema 6.1 a hipótese $\rho \geq 0$ é usada? (Dica: quatro vértices bastam; com três, o algoritmo desta seção, que relaxa também os arcos que chegam em S, ainda acerta. Tente os arcos (s, a), (s, b), (b, a), (a, c) de pesos $2, 3, -2, 1$.)

Exercício 103. Suponha que o teste da linha 6 do algoritmo de Dijkstra seja trocado por $|\overline{S}| > 1$, o que faria o laço ser executado $|V| - 1$ vezes em vez de $|V|$. O algoritmo continua correto? Justifique.

Exercício 104. Seja $D = (V, E, \rho)$ um grafo dirigido com $\rho : E \to [0, 1]$, em que $\rho(e)$ é a probabilidade de o arco e *não* falhar, e suponha que as falhas dos arcos são independentes. A probabilidade de que um caminho orientado P não falhe é $\prod_{e \in E(P)} \rho(e)$. Dados $s, t \in V$, escreva um algoritmo que determina um $(s \to t)$-caminho orientado com a maior probabilidade de não falhar e prove que ele está correto. (Dica: troque cada peso $\rho(e) > 0$ por $-\log \rho(e) \geq 0$ e descarte os arcos com $\rho(e) = 0$.)

Exercício 105. Seja $D = (V, E, \rho)$ um grafo dirigido com pesos $\rho : E \to \{0, 1, \dots, m\}$ nos arcos, $m \in \mathbb{N}$. Descreva uma fila de prioridades para o algoritmo de Dijkstra de modo que a complexidade de tempo para computar as distâncias seja $O(m|V| + |E|)$. (Dica: toda distância finita é no máximo $m(|V| - 1)$; use $m|V|$ listas duplamente ligadas, uma para cada valor de d, e um índice que só cresce; explique por que o índice nunca precisa voltar.)

Exercício 106. Dados um grafo dirigido G com pesos $\rho(e) \geq 0$ nos arcos e um vértice s, considere o algoritmo que inicia com $\mathrm{d}[v] \leftarrow \infty$ para todo vértice v e $\mathrm{d}[s] \leftarrow 0$ e, em seguida, repete $|V|$ vezes a instrução

para cada $(u, t) \in E(G)$ **faça** Relaxação(u, t).

Esse algoritmo determina $\mathrm{dist}_G(s, v)$ corretamente para todo v? Quantas repetições bastam? Qual a complexidade? O resultado vale se G tem arcos de peso negativo, mas nenhum circuito orientado de custo negativo? (Dica: indução no número de arcos de um caminho mínimo.)

Exercício 107. Mostre que, se numa rodada nenhum valor d[ ] é alterado, então nenhuma rodada seguinte altera valores e o laço da linha 6 pode ser interrompido. Que consequência isso tem para o teste de circuito negativo?

Exercício 108 (Propriedades da Relaxação). Seja $G = (V, E, \rho)$ um grafo dirigido com pesos e $s \in V$, e suponha que não há circuito negativo alcançável a partir de s. Suponha que d e p foram inicializados como no Algoritmo 33 (antes do laço da linha 6) e que depois disso foram feitas apenas execuções de Relaxação, em qualquer número e ordem. Prove que sempre valem:

(a) $\mathrm{d}[v] = \infty$ ou $\mathrm{d}[v]$ é o peso de um passeio de s a v, para todo $v \in V$;
(b) $\mathrm{d}[v] \geq \mathrm{dist}_G(s, v)$ para todo $v \in V$, e uma vez que vale a igualdade o valor de $\mathrm{d}[v]$ não muda mais;
(c) se $\mathrm{dist}_G(s, v) = \infty$, então $\mathrm{d}[v] = \infty$;
(d) se $s, \dots, u, w$ é um $(s \to w)$-caminho mínimo e $\mathrm{d}[u] = \mathrm{dist}_G(s, u)$, então depois da execução de Relaxação(u, w) tem-se $\mathrm{d}[w] = \mathrm{dist}_G(s, w)$;
(e) seja $s = v_0, v_1, \dots, v_k = w$ um caminho mínimo. Se ocorrem as relaxações de $(v_0, v_1), (v_1, v_2), \dots, (v_{k-1}, v_k)$, nessa ordem e possivelmente com outras relaxações intercaladas, então ao final da sequência $\mathrm{d}[w] = \mathrm{dist}_G(s, w)$.

Mostre com um exemplo que (b) pode falhar se há um circuito negativo alcançável a partir de s. Use (a)–(e) para provar o Lema 6.1 sem usar a Afirmação (b) da sua prova.

Exercício 109. Para $v \in V$, seja $\delta(s, v)$ o ínfimo dos pesos dos passeios de s a v; assim $\delta(s, v) = -\infty$ se, e somente se, existe um passeio de

s a v que passa por um circuito negativo, e $\delta(s,v) = \text{dist}_G(s,v)$ caso contrário. Modifique o algoritmo de Bellman–Ford para que, ao final, $\text{d}[v] = \delta(s,v)$ para todo $v \in V$. (*Dica:* depois das $|V|-1$ rodadas, faça mais $|V|$ rodadas em que $\text{d}[w] \leftarrow -\infty$ sempre que $\text{d}[u] = -\infty$ ou $\text{d}[w] > \text{d}[u] + \rho(u,w)$, para $(u,w) \in E$. Prove que isso funciona.)

Exercício 110. Suponha que existe um algoritmo $\mathcal{A}(G,u,v)$ que, em tempo polinomial, recebe um grafo dirigido G com pesos reais nos arcos (possivelmente negativos, inclusive com circuitos negativos) e devolve um $(u \to v)$-caminho de custo mínimo, ou informa que não existe caminho. Mostre como usar $\mathcal{A}$ para determinar em tempo polinomial se G tem um circuito orientado hamiltoniano[2]. (*Dica:* atribua peso -1 a todos os arcos e, para cada $(v,u) \in E(G)$, compute $\mathcal{A}(G,u,v)$ e veja se o custo é $-(|V|-1)$.) Por que o algoritmo de Bellman–Ford não é um algoritmo $\mathcal{A}$?

2: Circuito orientado hamiltoniano é um circuito orientado que passa por todos os vértices do grafo dirigido.

Como o problema de decidir se um grafo dirigido tem circuito hamiltoniano é NP-completo, a existência de $\mathcal{A}$ implicaria P = NP.

Exercício 111 (Exibindo um circuito negativo). Suponha que G tem um circuito negativo alcançável a partir de s, de modo que Bellman–Ford(G,s) devolve *falso*. Mostre como encontrar, em tempo $O(|V|)$ adicional, um circuito negativo. (*Dica:* seja (u,w) um arco que falha no teste final; execute Relaxação(u,w), siga o vetor p a partir de w por $|V|$ passos e mostre que o vértice atingido pertence a um circuito de p cujo peso é negativo.)

Exercício 112 (Grafos dirigidos acíclicos). Seja G um GDA com pesos arbitrários nos arcos e $s \in V(G)$. Mostre que, se os arcos são relaxados uma única vez, ordenados pela ordem topológica dos seus vértices de saída (todos os arcos que saem de um vértice antes dos arcos que saem de um vértice posterior), então ao final $\text{d}[v] = \text{dist}_G(s,v)$ para todo $v \in V(G)$. Qual o tempo total, incluindo o cálculo da ordem topológica (Seção 5.2)? (*Dica:* item (e) do Exercício 108.)

Exercício 113 (Fecho transitivo). Seja $G = (V,E)$ um grafo dirigido com $V = \{1,\ldots,n\}$. Considere a variante do Algoritmo 34 em que $\text{d}[i,j]$ é um valor booleano, inicializado com *verdadeiro* se $i = j$ ou $(i,j) \in E$, e *falso* caso contrário, e a atualização é $\text{d}[i,j] \leftarrow \text{d}[i,j] \vee (\text{d}[i,k] \wedge \text{d}[k,j])$. Prove que, ao final, $\text{d}[i,j]$ é *verdadeiro* se, e somente se, existe um $(i \to j)$-caminho orientado em G. Compare o tempo com o de calcular as componentes fortemente conexas (Seção 5.3).

Exercício 114 (Reconstrução dos caminhos). Suponha que G não tem circuito negativo. Prove que, para todo $i \neq j$ com $\text{d}[i,j] < \infty$, a sequência

$$j, \quad \text{p}[i,j], \quad \text{p}\big[i,\text{p}[i,j]\big], \ \ldots$$

chega a i sem repetir vértices e, lida de trás para frente, é um $(i \to j)$-caminho de peso $\text{d}[i,j]$. Escreva um algoritmo que imprime esse caminho em tempo $O(n)$.

Exercício 115 (A ordem dos laços importa). Mostre que, se trocarmos os laços de modo que k seja o laço mais interno, isto é, para cada i, para cada j, para cada k, o algoritmo pode calcular distâncias erradas mesmo sem circuitos negativos. (*Dica:* G com 4 vértices e os arcos $(2,3)$, $(3,4)$ e $(4,1)$, todos de peso 1.) Onde, na prova do Lema 6.2, é usado que k é o laço mais externo?

Exercício 116 (Quando usar cada algoritmo). Para grafos dirigidos com $|V| = n$ vértices, $|E| = \Theta(n)$ arcos e pesos $\rho \geq 0$, qual é mais rápido para calcular todas as distâncias: Floyd–Warshall ou n execuções de Dijkstra com *heap*? E para $|E| = \Theta(n^2)$? E se há arcos de peso negativo, mas nenhum circuito negativo? (*Dica:* execute Bellman–Ford uma vez a partir de um novo vértice s^* com arcos de peso 0 para todos os vértices e use os valores $\mathrm{d}[v]$ para trocar $\rho(u,v)$ por $\rho(u,v) + \mathrm{d}[u] - \mathrm{d}[v]$; mostre que os novos pesos são não negativos e que os caminhos mínimos não mudam.)

Exercício 117 (Diâmetro). Seja G um grafo dirigido fortemente conexo com pesos $\rho \geq 0$. A ***excentricidade*** de i é $\max_j \mathrm{dist}_G(i,j)$, o ***diâmetro*** de G é a maior excentricidade e o ***raio*** é a menor. Mostre como calcular o diâmetro, o raio e um vértice de excentricidade mínima em tempo $O(n^3)$, e que depois de obtida a matriz d isso custa só $O(n^2)$.